

Lab 07 — Queues and the Ring Buffer

DSA27 Lab Manual · Week 7 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026 · Week 7

Contents

Part 0 — Before you start	2
Part 1 — <code>SlowQueue</code> : the obvious design	4
Part 2 — The ring on paper	6
Part 3 — <code>CircularQueue.enqueue</code>	7
Part 4 — <code>CircularQueue.dequeue</code>	9
Part 5 — Full or empty?	11
Part 6 — Challenge: grow when full	11
Part 7 — Measure it	14
Part 8 — Exercises at a glance	15
Part 9 — Take-home practice (not graded)	16
Part 10 — Bridge to next week's lecture: searching	16
Summary	17
Answers to the checkpoints	18

How to use this lab. Read the matching section of Lecture 07 before each part. **Draw before you code:** every ring in this lab fits on a scrap of paper, and a ring you have traced by hand is one you can write. At each **Checkpoint**, write your prediction down first (answers at the end). Build `dsa/queue.py` one method at a time, and run the named tests after each part — not all of them at the end.

Duration	One 2-hour lab session, plus about 3 hours at home
----------	--

You will write	dsa/queue.py — SlowQueue.enqueue, SlowQueue.dequeue, CircularQueue.enqueue, CircularQueue.dequeue, and the challenge: growing a full ring
Graded by	the queue half of tests/test_stack_queue.py: pytest tests/test_stack_queue.py -v -k "fifo or dequeue or circular" (6 tests)
Connects to	Lecture 07 — Queues; Lecture 04 (your DynamicArray); Lecture 08 — Searching

What you will be able to do

1. write a FIFO queue on your own DynamicArray, and say exactly which of its two operations is $O(n)$ and why;
 2. trace a ring buffer on paper — block, head and size after every operation — and check the trace against the real object;
 3. compute the back of a ring as $(\text{head} + \text{size}) \% \text{capacity}$, and wrap head with the same %;
 4. explain why `head == tail` cannot tell a full ring from an empty one, and how `_size` settles it;
 5. grow a full ring without scrambling it, by copying in **queue order**;
 6. write your own test for behaviour the provided tests never reach;
 7. measure the cost of one dequeue at several sizes, and read $O(n)$ against $O(1)$ off the plot.
-

Part 0 — Before you start

0.1 Environment and prerequisite

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
pytest -m "not challenge" -q          # the environment check: must pass
pytest tests/test_dynamic_array.py -q # your Week 4 DynamicArray: 19 passed
```

SlowQueue stands on **your** DynamicArray: its enqueue is your append and its dequeue is your `pop(0)`. If `test_dynamic_array.py` fails, fix that first — a queue built on a broken array fails in ways that look like queue bugs. In particular, `test_pop_on_empty_raises` must pass: your `pop` raises `IndexError` on an empty array, and this lab relies on it.

CircularQueue does **not** use DynamicArray. It stands directly on the course Array (`dsa/array.py`) — the fixed block of slots from Lecture 02, which cannot grow and has no negative indices.

0.2 Read the skeleton: what is given, what is yours

Open `dsa/queue.py`. Read all of it before you write anything.

Class	Given to you	Yours
SlowQueue	<code>__init__(values=())</code> — builds <code>self._items</code> , a <code>DynamicArray</code> ; <code>__len__</code>	<code>enqueue</code> , <code>dequeue</code>
CircularQueue	<code>__init__(capacity=8)</code> — <code>_block</code> , <code>_capacity</code> , <code>_head</code> , <code>_size</code> ; <code>is_empty</code> , <code>is_full</code> , <code>__len__</code> , <code>__iter__</code> , <code>__repr__</code>	<code>enqueue</code> (and the growth challenge in its docstring), <code>dequeue</code>

Read `CircularQueue.__iter__` twice. It already contains the ring arithmetic you need:

```
def __iter__(self):
    for offset in range(self._size):
        yield self._block[(self._head + offset) % self._capacity]
```

The i -th item of the queue, counting from the front, lives in slot $(\text{head} + i) \% \text{capacity}$. Everything in this lab is that one line, used in three places.

Notice also what is **not** stored: there is no `_tail`. As the class docstring says, the slot the next enqueue writes to — the tail, $(\text{head} + \text{size}) \% \text{capacity}$ — is computed each time, not stored (Lecture 07, “Idea 2: wrap around”).

0.3 The storage rule, this week

A `dsa/` structure stores its data only in the course `Array`, in node objects, or in another `dsa/` structure — never in a Python list, dict, set or `collections.deque` (Lecture 02). Here:

- `SlowQueue` keeps everything in `self._items`, your `DynamicArray`. Do not add a list beside it.
- `CircularQueue` keeps everything in `self._block`, an `Array`. When it grows, the new block is another `Array` — not a list you convert later.
- Lists are still fine as **inputs and outputs**: `list(q)` in a test, or `SlowQueue([5, 6])`, is not storage.

0.4 The tests, and one warning about -k

The queue tests live in the same file as Week 6’s stack tests. Four test functions cover the queues; two are run once for each class, so `pytest` reports **six** tests:

Test	Checks
<code>test_queue_is_fifo[SlowQueue],</code> <code>[CircularQueue]</code>	<code>enqueue 1, 2, 3; dequeue gives 1 then 2; len is 1</code>

Test	Checks
<code>test_dequeue_on_empty_raises[SlowQueue]</code>	dequeuing a new, empty queue raises <code>IndexError</code>
<code>test_circular_queue_wraps_around</code>	capacity 3: fill, dequeue once, enqueue into the freed slot
<code>test_circular_queue_interleaved_operations</code>	capacity 4: ten enqueues and eight dequeues, wrapping twice

The command that runs exactly these six is:

```
pytest tests/test_stack_queue.py -v -k "fifo or dequeue or circular"
```

It is the same command as Lecture 07's "Exercises" slide. Why such a long filter, and not simply `-k queue`? `-k` matches against the file name as well as the test name, and the file is `test_stack_queue.py`. So a filter that contains `queue` (or `stack`) matches **every** test in the file: `-k queue` selects all 18, stack tests included. That is harmless if your Week 6 Stack passes, but it is not "the queue half". The smaller filters used in this lab avoid both words:

Filter	Selects
<code>-k SlowQueue</code>	the 2 <code>SlowQueue</code> tests
<code>-k circular</code>	the 4 <code>CircularQueue</code> tests
<code>-k wraps</code>	<code>test_circular_queue_wraps_around</code> only
<code>-k interleaved</code>	<code>test_circular_queue_interleaved_operations</code> only

Right now all six fail with `NotImplementedError`. That is the starting line.

Part 1 — `SlowQueue`: the obvious design

1.1 The idea

Lecture 07, "The obvious design": the back of the queue is the end of the `DynamicArray` and the front is index 0. Enqueue is `append` — amortised $O(1)$. Dequeue is `pop(0)` — every remaining element shifts one slot left, $O(n)$. The class is named for its flaw, because in Part 7 you will measure it.

Could you swap the ends? Then dequeue would be `pop()` at the end, $O(1)$, but enqueue would be `insert_at(0, x)`, $O(n)$. On a plain array one of the two operations is always at the front. That is the whole difference from Week 6: a stack only ever touches one end.

1.2 Draw it

Draw a `DynamicArray` holding 5 6 7 in slots 0, 1, 2. Dequeue once. Draw the block again, with an arrow for every element that moved. How many arrows? For a queue of 1,000 items, how many?

1.3 Checkpoint

Checkpoint 1. Once your *SlowQueue* works, what does this print? And how many elements does that one *dequeue* move?

```
from dsa.queue import SlowQueue
q = SlowQueue([5, 6])
q.enqueue(7)
print(q.dequeue(), len(q))
print(q._items)
```

1.4 Write it

enqueue(value) — one call on `self._items`.

dequeue():

1. If the queue is empty, raise `IndexError` — the docstring's last line, "Raises `IndexError` when empty", and `test_dequeue_on_empty_raises` checks it.
2. Remove the item at index 0 and **return** it.

Hints:

- Your `DynamicArray.pop` already raises `IndexError` on an empty array, so the test passes even without step 1. Write the check anyway, with a message that names the queue — `IndexError("dequeue from empty queue")` — so that an error from your queue does not look like an error from the array underneath.
- Do not reach into `self._items._block`. `SlowQueue` uses the `DynamicArray` through its public methods only — that is the ADT boundary from Lecture 01.

1.5 Test it

```
pytest tests/test_stack_queue.py -v -k SlowQueue
```

Two tests: `test_queue_is_fifo[SlowQueue]` and `test_dequeue_on_empty_raises[SlowQueue]`.

1.6 When it fails

`SlowQueue` has no `__repr__`, so `pytest` shows it as `<dsa.queue.SlowQueue object at 0x...>`. Read the assert line, not the object.

What you see	Cause	Fix
<code>assert 3 == 1 in</code> <code>test_queue_is_fifo[SlowQueue]</code>	<code>self._items.pop()</code> — the test item leaves first: a stack, not a queue (W7-B1)	<code>pop(0)</code>
<code>assert None == 1</code>	you called <code>pop(0)</code> but forgot <code>return</code>	return what <code>pop</code> gives you

What you see	Cause	Fix
Failed: DID NOT RAISE IndexError in test_dequeue_on_empty_raises	<code>if len(self._items) == 0: return None</code> — a quiet <code>None</code> instead of an error	<code>raise IndexError(...);</code> None could be a real value in the queue

Part 2 — The ring on paper

Do this part with a pencil before you touch `CircularQueue`.

2.1 The idea

Lecture 07, “Idea 1” and “Idea 2”. Instead of shifting everything left, keep a head index and move **it**. On its own that leaves dead slots behind the front; the ring fixes it by treating the slot after the last one as slot 0. Three numbers describe the whole state: head (the slot of the front item), size (how many items), and capacity (how many slots). The back is computed:

```
tail = (head + size) % capacity      # where the NEXT enqueue writes
```

2.2 Draw it: the lecture’s trace

Copy this table from Lecture 07, “Trace: capacity 4”, and check every row with the formula. Before “enqueue E”: $\text{tail} = (2 + 2) \% 4 = 0$ — the write wraps round to slot 0.

Operation	Block	head	size	Returns
start	_ _ _ _	0	0	
enqueue A, B, C	A B C _	0	3	
dequeue	_ B C _	1	2	A
dequeue	_ _ C _	2	1	B
enqueue D	_ _ C D	2	2	
enqueue E	E _ C D	2	3	
enqueue F	E F C D	2	4	
dequeue	E F _ D	3	3	C

Read the last block as a ring from head: D (slot 3), E (slot 0), F (slot 1). No element ever moved.

2.3 Checkpoint

Checkpoint 2. Trace `q = CircularQueue(capacity=4)` through the operations below. After **each** one, write the block, head and size. What does each dequeue return, and what are `List(q)`, `q._block`, `q._head` and `q._size` at the end?

```
enqueue 1, enqueue 2, enqueue 3, enqueue 4,
dequeue, dequeue, dequeue,
enqueue 5, enqueue 6,
dequeue
```

This trace makes **both** indices wrap: the back wraps on “enqueue 5”, and head wraps on the last dequeue.

2.4 Check a trace against the real object

Once Parts 3 and 4 are done, you can check any paper trace at the REPL. The fields are private by convention, not by force — reading them to debug is fine; writing to them from outside the class is not.

```
>>> from dsa.queue import CircularQueue
>>> q = CircularQueue(capacity=4)
>>> for v in 'ABC':
...     q.enqueue(v)
...
>>> q.dequeue(), q.dequeue()
('A', 'B')
>>> q._block, q._head, q._size
(Array([None, None, 'C', None]), 2, 1)
>>> list(q)
['C']
```

To **see** the ring, draw its block with `viz.draw.draw_array`. The front slot is highlighted, and the rest of the queue is shown in green:

```
import matplotlib.pyplot as plt
from viz.draw import draw_array

def show(q):
    cells = ['' if v is None else v for v in q._block]
    rest = [(q._head + i) % q._capacity for i in range(1, len(q))]
    tail = (q._head + len(q)) % q._capacity
    draw_array(cells, highlight=q._head if len(q) else None, done=rest,
               title=f'head={q._head} size={len(q)} tail={tail}')
    plt.show()
```

After “enqueue E” in the lecture’s trace, `show(q)` draws E _ C D with slot 2 highlighted, slots 3 and 0 green, and the title `head=2 size=3 tail=1`. Put `show` in the 07-queues notebook and call it after every step of Checkpoint 2.

Part 3 — CircularQueue.enqueue

3.1 The idea

Lecture 07, "The two operations":

1. full? — decide what to do (below);
2. write value at $(\text{head} + \text{size}) \% \text{capacity}$;
3. $\text{size} += 1$.

Nothing moves, and head does not change: enqueue only touches the back.

What to do when full. Start with the simple, honest choice: `raise IndexError("queue is full")`. The provided tests never overfill a ring, so this passes them. Part 6 replaces it with growth.

3.2 Draw it

A ring of capacity 5 holds d in slot 3 and e in slot 4. Draw it. Mark head. Now mark the slot the next enqueue writes to.

3.3 Checkpoint

Checkpoint 3. A *CircularQueue* has capacity 5, *head* = 3 and *size* = 2.

- (a) Which slots hold the queue, front first?
- (b) Which slot does the next enqueue write to?
- (c) A student writes `self._block[self._head + self._size] = value`, with no `%`. What happens on that enqueue?

3.4 Write it

Write the three steps above. Hints:

- Test fullness with `self._size == self._capacity` — or call the given `is_full()`. **Not** with `tail == head` (Part 5 explains why).
- Compute the slot **after** any full check, from the current fields.
- Keep it $O(1)$: no loop in enqueue.

3.5 Test it

`dequeue` is not written yet, so every *CircularQueue* test still fails at its first `dequeue`. Check `enqueue` at the REPL instead — this is exactly what the given `__iter__` is for:

```
>>> q = CircularQueue(capacity=3)
>>> for v in (1, 2, 3):
...     q.enqueue(v)
...
>>> list(q), q.is_full(), q._block
([1, 2, 3], True, Array([1, 2, 3]))
>>> q.enqueue(4)
Traceback (most recent call last):
```

...

`IndexError: queue is full`

3.6 When it fails

These are the symptoms once `dequeue` is written too, when you run `pytest tests/test_stack_queue.py -v -k circular`:

What you see	Cause	Fix
<code>IndexError: Array index 3 out of range for length 3 (valid: 0..2; no negative indices) in test_circular_queue_wraps_block</code> , raised from <code>dsa/array.py</code>	the slot is <code>head + size</code> with no % capacity: after one <code>dequeue</code> , the fourth value is written to slot 3 of a 3-slot block	<code>(self._head + self._size) % self._capacity</code>
<code>IndexError: queue is full on the first enqueue</code> , in three tests your own empty-queue <code>IndexError</code> (for example <code>dequeue from empty queue</code>) in <code>test_queue_is_fifo[CircularQueue]</code> , although three values were enqueued	fullness tested as <code>tail == head</code> — also true for an empty ring (W7-B3) forgot <code>self._size += 1</code> : every enqueue writes to the same slot, and the queue stays empty	<code>test self._size == self._capacity</code> <code>increment _size after the write</code>
<code>assert 3 == 1 in test_queue_is_fifo[CircularQueue]</code>	writing at <code>self._head</code> instead of the tail: each enqueue overwrites the front	<code>write at (head + size) % capacity</code>

Part 4 — `CircularQueue.dequeue`

4.1 The idea

Lecture 07, “The two operations”:

1. empty? — raise `IndexError`;
2. read the value at `head`; clear that slot;
3. `head = (head + 1) % capacity`; `size -= 1`.

Then return the value.

Why clear the slot? The queue is correct without it: the slot is outside the live part of the ring and will be overwritten later. But until then the block keeps a reference to the dequeued object, and Python cannot free it. It is the same care as your `DynamicArray.pop` setting the vacated slot to `None`. No test checks it; the TA may ask about it.

4.2 Draw it

Take the lecture's trace at "enqueue F" (E F C D, head 2, size 4). Dequeue four times on paper. Write head after each: it should go 3, 0, 1, 2 — and the block should end empty.

4.3 Checkpoint

Checkpoint 4. A student writes step 3 as `self._head += 1`, with no %, and everything else correctly. Of the four `CircularQueue` tests selected by `-k circular`, which fail, and with what message? Why does `test_circular_queue_wraps_around` not catch it?

4.4 Write it

Write the steps above. Hints:

- Read the value into a local variable **before** you clear the slot.
- Check emptiness with `self._size == 0` (or `is_empty()`), first thing.
- No loop. If you find yourself writing for in dequeue, stop and redraw.

4.5 Test it

```
pytest tests/test_stack_queue.py -v -k circular
```

Four tests: `test_queue_is_fifo[CircularQueue]`, `test_dequeue_on_empty_raises[CircularQueue]`, `test_circular_queue_wraps_around` and `test_circular_queue_interleaved_operations`. Then the whole queue half:

```
pytest tests/test_stack_queue.py -v -k "fifo or dequeue or circular"
```

Six passed. Now go back to Checkpoint 2 and check your paper trace against the real object, as in Part 2.4.

4.6 When it fails

What you see	Cause	Fix
<code>IndexError: Array index 4 out of range for length 4 (valid: 0..3; no negative indices) in test_circular_queue_interleaved_operations only, raised by __getitem__</code> <code>assert None == 1 in three tests</code> <code>Failed: DID NOT RAISE IndexError in test_dequeue_on_empty_raises</code>	head never wraps (Checkpoint 4, W7-B2) the slot was cleared before it was read no emptiness check: an empty ring returns the <code>None</code> instead of <code>0</code> and <code>size</code> to <code>-1</code>	<pre>self._head = (self._head + 1) % self._capacity</pre> <pre>read into a local first, then clear</pre> <pre>if self._size == 0: raise IndexError(...)</pre>

What you see	Cause	Fix
<code>assert 3 == 1</code> where <code>3 = len(CircularQueue([3, None, None]))</code>	forgot <code>self._size -= 1</code> : the queue never shrinks, and <code>__iter__</code> now reads cleared slots	decrement <code>_size</code>

Part 5 — Full or empty?

5.1 The idea

Lecture 07, “Full or empty?”. Compute the tail at the start of a queue’s life and again when it is full. In both, `tail == head`: the next write would go to the front’s slot. A ring that stored only head and tail could not tell the two apart. `dsa/queue.py` stores `_size` instead of a tail, which is why `is_empty (size == 0)` and `is_full (size == capacity)` are one-liners. The other two fixes — leave one slot unused, or keep a flag — also work; having none is the bug.

5.2 Checkpoint

Checkpoint 5. For each ring, give head, the computed tail, `is_empty()` and `is_full()`:

```
a = CircularQueue(3)                                # new

b = CircularQueue(3)
for v in (1, 2, 3):
    b.enqueue(v)
b.dequeue()
b.enqueue(4)
```

What is `b._block`? What would `head == tail` say about each?

This is also the exam’s favourite bug (W7-B3): an enqueue that tests `tail == head` for fullness refuses the very first value.

Part 6 — Challenge: grow when full

6.1 Why bother

The docstring of `CircularQueue.enqueue` calls it a challenge, and the tests do not require it. Do it anyway: Week 11’s `level_order` and Week 14’s `bfs` use your `CircularQueue`, and neither knows in advance how many items it will hold.

The plan is Lecture 04’s: when full, allocate a bigger Array — double the capacity — copy, and switch. Doubling keeps enqueue **amortised** $O(1)$: over n enqueues the copies total less than $2n$.

6.2 The trap: the naive copy

Lecture 07, “Growing: the trap”. The obvious copy is the one you wrote in `DynamicArray._resize`: slot i of the old block to slot i of the new. On a ring that has **wrapped**, that is wrong.

Take the full ring from the lecture’s trace after “enqueue F”: block E F C D, head 2, size 4. The naive copy gives the block E F C D _ _ _ _, with head still 2 and capacity now 8. Reading from head: C, D, then two empty slots. E and F are “behind” the front and lost.

Checkpoint 6. A student grows with the naive copy (slot i to slot i , head unchanged, capacity updated to 8), then enqueues G into the grown ring above. Where is G written, and what is `List(q)`?

If the ring has **not** wrapped yet (head = 0), the naive copy happens to work. That is exactly why this bug survives casual testing — and why the test you write in 6.4 wraps first.

6.3 Write it: unroll in queue order

Write a helper `_grow(self, capacity)` and call it from `enqueue` when the ring is full, instead of raising.

1. Make a new Array of the new capacity.
2. For i from 0 to `size - 1`: the i -th item of the queue lives at $(\text{head} + i) \% \text{old_capacity}$ — the same formula as `__iter__`. Put it in slot i of the new block.
3. Switch: the block becomes the new one, head becomes **0**, and capacity becomes the new capacity. Three fields, all three.

Hints:

- **Grow first, then compute the slot.** In `enqueue`, the full check and the growth come before $(\text{head} + \text{size}) \% \text{capacity}$. Compute the slot from the old capacity and you write into the wrong place in the new block.
- **Capacity 0.** `CircularQueue(0)` is legal. Doubling 0 gives 0, and the next % is `ZeroDivisionError: integer modulo by zero`. Grow to $\max(1, 2 * \text{capacity})$.
- size does not change during growth: the same items, in a bigger block.

6.4 Test it: write your own test

The provided tests cannot check this — they never overfill a ring. So write the test yourself. Create `test_my_queue.py` in the DSA27 folder (**not** in `tests/`, which stays exactly as it was given):

```
from dsa.queue import CircularQueue

def test_grows_after_wrapping():
    q = CircularQueue(capacity=4)
    for value in "ABC":
        q.enqueue(value)
    q.dequeue()
```

```

q.dequeue()                # head is now 2
for value in "DEF":
    q.enqueue(value)        # E and F wrap round to slots 0 and 1
assert q.is_full()
q.enqueue("G")              # must grow, not raise
assert list(q) == ["C", "D", "E", "F", "G"]
assert [q.dequeue() for _ in range(5)] == ["C", "D", "E", "F", "G"]
assert q.is_empty()

def test_grows_many_times_in_order():
    q = CircularQueue(capacity=2)
    expected = []            # a plain list is fine here: this is a test
    for value in range(100):
        q.enqueue(value)
        expected.append(value)
        if value % 3 == 0:
            assert q.dequeue() == expected.pop(0)
    assert list(q) == expected

pytest test_my_queue.py -v
pytest tests/test_stack_queue.py -v -k "fifo or dequeue or circular"
    
```

The first test is the lecture's trace plus one enqueue; the second grows a ring six times, from 2 to 128 slots, with dequeues in between — so head is never 0 when the ring fills, and every one of those growths happens on a wrapped ring. Run the provided tests again afterwards: growth must not break anything that worked.

6.5 When it fails

What you see in

test_grows_after_wrapping

Cause

IndexError: queue is full

enqueue still raises instead of calling `_grow`
the naive copy: slot `i` to slot `i`, head kept

assert ['C', 'D', None, None, 'G']

== ['C', 'D', 'E', 'F', 'G']

copied in queue order, but forgot head = 0

assert ['E', 'F', None, None, 'G']

== [...]

assert ['G', 'D', 'E', 'F', 'G'] ==

forgot to update capacity: every % still wraps
at 4, so G overwrote slot 0 of the new block
the slot was computed **before** growing, with
the old capacity

[...]

assert ['C', 'D', 'G', 'F', None] ==

[...]

Part 7 — Measure it

7.1 What the lecture claims

Lecture 07, "Measured": one `SlowQueue` dequeue costs $O(n)$ — a straight line of slope 1 on a log-log plot — while one `CircularQueue` dequeue costs $O(1)$, a flat line. You can now check that on your own code.

7.2 The snippet

Run this in `notebooks/07-queues.ipynb` (the notebook's "Try it" cell imports `measure` and `plot_growth` for you):

```
from dsa.queue import CircularQueue, SlowQueue
from viz.complexity import measure, plot_growth

OPS = 200                                # dequeues timed per run

def make_slow(n):
    return SlowQueue(range(n))            # n items, built before the clock starts

def make_ring(n):
    q = CircularQueue(capacity=n)
    for i in range(n):
        q.enqueue(i)
    return q

def churn(q):
    for _ in range(OPS):                  # size stays n: one out, one in
        q.enqueue(q.dequeue())

sizes = [250, 500, 1000, 2000, 4000]
slow = measure(churn, sizes, make_slow)
ring = measure(churn, sizes, make_ring)

def per_op(result):
    ns, secs = result
    return ns, [s / OPS for s in secs]

plot_growth({"SlowQueue": per_op(slow), "CircularQueue": per_op(ring)},
            reference=["n", "1"], loglog=True, title="One dequeue, queue of n items")
```

Points to notice:

- `measure` builds each input with `make_slow` or `make_ring` **outside** the timing, so only the 200 dequeue-and-enqueue pairs are timed.
- One dequeue on the ring is far too quick to time on its own, so `churn` does 200 and `per_op`

divides. Each pair enqueues what it dequeued, so the queue holds n items throughout — the ring never needs to grow.

- Sizes from 250 to 4,000 keep the `SlowQueue` runs short: measure runs each size four times (one warm-up, then three timed), and the whole cell takes well under a minute. Every doubling of n doubles the `SlowQueue` part of that wait, so add larger sizes one at a time.

7.3 What to expect

On a log–log plot, the `SlowQueue` points lie close to the dashed $O(n)$ line, rising from a fraction of a millisecond to several milliseconds per dequeue. The `CircularQueue` points sit at a few microseconds — about a hundred times lower at $n = 250$, and over a thousand times lower at $n = 4,000$ — and do not climb as n grows. Timings vary with your machine and with what else is running; single points wobble, the **shape** does not.

Checkpoint 7. *Without running anything: when n doubles from 2,000 to 4,000, by roughly what factor should the time per `SlowQueue` dequeue grow? Per `CircularQueue` dequeue? And if one `SlowQueue` dequeue at $n = 4,000$ took 5 ms, roughly how long would one take at $n = 1,000,000$?*

Homework item 2 of Lecture 07 asks for the **total** cost — n enqueues followed by n dequeues — where the `SlowQueue` is $O(n^2)$ and the ring $O(n)$. That version is much slower for the `SlowQueue`; keep n at 4,000 or below there too.

Part 8 — Exercises at a glance

Method	Target cost	The trap	Tests
<code>SlowQueue.enqueue</code>	amortised $O(1)$	none — one append	-k <code>SlowQueue</code>
<code>SlowQueue.dequeue</code>	$O(n)$, on purpose	<code>pop()</code> instead of <code>pop(0)</code> ; returning <code>None</code> when empty	-k <code>SlowQueue</code>
<code>CircularQueue.enqueue</code>	$O(1)$	the % on the tail; testing full by <code>tail == head</code> ; forgetting <code>size += 1</code>	-k <code>circular</code>
<code>CircularQueue.dequeue</code>	$O(1)$	head must wrap; read before clearing; <code>IndexError</code> when empty	-k <code>circular</code>
<i>challenge: grow when full</i>	amortised $O(1)$	copy in queue order; then <code>head = 0</code> and the new capacity	your <code>test_my_queue.py</code>

All six provided tests:

```
pytest tests/test_stack_queue.py -v -k "fifo or dequeue or circular"
```

Part 9 — Take-home practice (not graded)

The Week 7 question bank is docs/question-bank/week07-questions.md, with answers in week07-answers.md. Do these after the lab:

1. **W7-S1** — draw the block, head and size of a CircularQueue(capacity=5) through ten operations. Check your drawing with show(q) from Part 2.4.
2. **W7-T1** — a capacity-4 trace that prints is_full() and list(q). Predict, then run it.
3. **W7-S2** — draw what the missing % of Checkpoint 3 does inside test_circular_queue_wraps_around.
4. The coding problems **W7-C1 to W7-C5** in practice/week07.py — a queue from two stacks, Josephus, moving averages on a CircularQueue(k), reversing the first k values, and the binary numbers generated breadth-first. They use your CircularQueue and your Week 6 Stack as working storage:

```
pytest tests/test_practice_week07.py -v
```

Worked solutions are in solutions/dsa/queue.py and solutions/practice/week07.py — for **after** you have tried (solutions/README.md says how to use them without wasting them).
pytest --solutions tests/test_stack_queue.py -v runs the tests on the reference, if you are unsure whether a failure is your code or your reading of the test.

Part 10 — Bridge to next week's lecture: searching

Lecture 08 is searching, and its exercises are in dsa/searching.py, graded by tests/test_searching.py. This week you made a structure fast by changing how it is **stored**. Next week you make a question fast — “is x in here?” — by changing how the data is **ordered**.

10.1 Halving

Linear search looks at every value: $O(n)$. Binary search looks at the middle of a **sorted** array and throws away the half the target cannot be in. How many halvings take a million values down to one?

```
>>> n, halvings = 1_000_000, 0
>>> while n > 1:
...     n //= 2
...     halvings += 1
...
>>> halvings
19
```

Nineteen halvings, and one more comparison to settle the last value: at most 20 comparisons for a million items (Lecture 08, "How many comparisons?"). The `SlowQueue` of Part 7 did a million moves for **one** dequeue at that size.

10.2 The precondition

Binary search works **only** because the values are sorted. Nothing checks this for you — checking would itself cost $O(n)$. Python's `bisect` module does the halving; watch what happens when the promise is broken:

```
>>> from bisect import bisect_left
>>> bisect_left([3, 8, 12, 19, 24, 31], 19)    # sorted: 19 is at index 3
3
>>> unsorted = [24, 3, 31, 8, 19, 12]
```

Checkpoint 8. Predict `bisect_left(unsorted, 19)`. (It returns the index where 19 would be inserted to keep the list sorted — assuming it is sorted.)

No error, no warning — just a wrong answer. In `dsa/searching.py`, write `linear_search` first, then `binary_search` from the steps in Lecture 08, "The algorithm, and the promise it keeps":

```
pytest tests/test_searching.py -v -k "unsorted or (binary_search and not recursive)"
```

That filter selects 12 of the file's 56 tests: the linear-search test on unsorted data and the eleven that check the loop form of `binary_search`. (A plainer `-k "linear or binary"` also picks up jump, exponential and interpolation search, through `test_agrees_with_linear_search_on_random_data`.)

Summary

Idea	The one line to keep
Queue	FIFO: enqueue at the back, dequeue at the front; empty dequeue raises <code>IndexError</code> .
<code>SlowQueue</code>	<code>append</code> and <code>pop(0)</code> on your <code>DynamicArray</code> : correct, and $O(n)$ per dequeue.
The ring	head moves, nothing shifts; the i -th item is at $(\text{head} + i) \% \text{capacity}$.
Enqueue	write at $(\text{head} + \text{size}) \% \text{capacity}$, then <code>size += 1</code> . $O(1)$.
Dequeue	read at head, clear it, <code>head = (head + 1) \% capacity</code> , <code>size -= 1</code> . $O(1)$.
Full or empty	<code>head == tail</code> means either; <code>size</code> tells them apart.
Growing	copy in queue order, then <code>head = 0</code> and the new capacity; double, for amortised $O(1)$.

Idea	The one line to keep
Your own test	the provided tests never overfill a ring — a test that wraps first is yours to write.
Measured	per dequeue: <code>SlowQueue</code> a line of slope 1, <code>CircularQueue</code> flat.
Next week	binary search: $O(\log n)$, but only on sorted data.

Answers to the checkpoints

Checkpoint 1.

5 2

```
DynamicArray([6, 7])
```

The queue was 5 6 7; the front, 5, leaves and two items remain. `pop(0)` moved **two** elements, 6 and 7, one slot left each. With 1,000 items it would move 999: that is the $O(n)$.

Checkpoint 2.

Operation	Block	head	size	Returns
enqueue 1, 2, 3, 4	1 2 3 4	0	4	(full)
dequeue	_ 2 3 4	1	3	1
dequeue	_ _ 3 4	2	2	2
dequeue	_ _ _ 4	3	1	3
enqueue 5	5 _ _ 4	3	2	
enqueue 6	5 6 _ 4	3	3	
dequeue	5 6 _ _	0	2	4

At the end `list(q)` is `[5, 6]`, `q._block` is `Array([5, 6, None, None])`, `q._head` is 0 and `q._size` is 2. "Enqueue 5" wrote to $(3 + 1) \% 4 = 0$; the last dequeue moved head from 3 to $(3 + 1) \% 4 = 0$.

Checkpoint 3. (a) Slots 3 and 4, in that order. (b) Slot $(3 + 2) \% 5 = 0$. (c) It writes to slot 5 of a 5-slot `Array`, which raises `IndexError: Array index 5 out of range for length 5 (valid: 0..4; no negative indices)` — although the ring has three free slots.

Checkpoint 4. Only `test_circular_queue_interleaved_operations` fails, with `IndexError: Array index 4 out of range for length 4 (valid: 0..3; no negative indices)` on a dequeue: the fifth dequeue reads `block[4]`. `test_circular_queue_wraps_around` dequeues only once, so head only reaches 1; its `list(q)` goes through the given `__iter__`, which applies `% capacity` itself. The other two tests never come near the end of the block.

Checkpoint 5. a: head 0, tail $(0 + 0) \% 3 = 0$, `is_empty()` is `True`, `is_full()` is `False`. b: `b._block` is `Array([4, 2, 3])`, head 1, tail $(1 + 3) \% 3 = 1$, `is_empty()` is `False`, `is_full()`

is True. In both, `head == tail` — so that test alone would call both of them empty (or both full). The size is what differs: 0 against 3.

Checkpoint 6. After the naive growth the block is E F C D _ _ _ , head 2, capacity 8, size 4. G is written to $(2 + 4) \% 8 = \text{slot 6}$, and `list(q)` reads slots 2 to 6: ['C', 'D', None, None, 'G']. E and F are still in the block, in slots 0 and 1, but outside the part the queue reads.

Checkpoint 7. About **2** for the `SlowQueue` — twice as many elements shift per dequeue — and about **1** for the ring: no change. Single measurements wobble, so expect something near 2, not exactly 2. At $n = 1,000,000$, 250 times 4,000, one `SlowQueue` dequeue would take about 250×5 ms, roughly **1.25 seconds** — for one operation that the ring does in microseconds.

Checkpoint 8. 6 — “insert at the end”, which means “19 is not here”, although 19 is at index 4. The halving compared 19 with values that were not in order and threw away the half that held it. Binary search on unsorted data gives wrong answers silently; the sorted order is the caller’s promise.